

The Runic System

Technical Guide

A plain-language explanation of how the classroom rune-tracking app works, plus the technical reference a future Claude session needs to safely extend it.

Covers: `runic-system-cloud.html` + `Code.gs` + Google Sheet backend
Also documents: `runic-system.html`, the offline single-device fallback version

Read this first if you are Claude

This document is written to be pasted into a future conversation as context before making changes to the Runic System app. Section 9 ("Notes for future Claude sessions") is aimed specifically at you: it lists where each piece of logic lives, what must stay in sync between files, and the deployment steps a change requires. Skim Sections 2–5 for the architecture and data model before editing anything, since the frontend and backend duplicate some data on purpose (see Section 5.4).

1. What this app is, in plain terms

The Runic System is a classroom reward tool for a Dungeons & Dragons campaign. Students earn **runes** — twelve different kinds, each with its own rarity — by redeeming one-time codes the teacher hands out at quest milestones. Runes can then be combined into **runewords**: magic items usable in the tabletop game (a ring, a sword, a pair of boots, and so on), each requiring a specific combination of runes to craft.

The app is a website with three main things a student can do, plus a teacher-only control panel:

- **Roll** — redeem a one-time milestone code for three random runes.
- **Transform** — trade in a combination of lower-value runes to forge a specific higher-value rune.
- **Craft** — spend a specific set of runes to permanently forge a named magic item (a "runeword").
- **Teacher tools** — generate milestone codes, hand-grant rare "God Runes", see every student's progress, and reset a forgotten PIN.

Every student's rune count lives in one shared, always-up-to-date place, so it doesn't matter whether a student checks their runes on a classroom Chromebook, their own phone, or the teacher's laptop — everyone sees the same numbers.

2. How the pieces fit together

There is no traditional "app server." Instead, three ordinary pieces of infrastructure work together:

Piece	What it is	Where it lives
Frontend	A single self-contained HTML file with all its CSS and JavaScript inline. This is what a student's browser actually loads and displays.	Hosted as a plain static file, e.g. on illswap.net.
Backend	A small program (Google Apps Script) that receives requests from the frontend, applies the game rules, and reads/writes the spreadsheet.	Runs inside Google's infrastructure, attached to a Google Sheet.
Database	An ordinary Google Sheet with three tabs (Students, Pins, Config) used as a simple row-based database.	Lives in the teacher's Google Drive.

The request flow for, say, rolling for runes, looks like this:

Student's browser	Google Apps Script	Google Sheet
-----	-----	-----
1. Click "Roll"		
2. fetch() POST ----->	2. doPost() receives it	
	3. Checks the student's PIN --->	reads Students tab
	4. Checks the milestone code -->	reads Pins tab
	5. Rolls 3 random runes	
	6. Writes new totals ----->	writes Students tab
	7. Marks the code as used ----->	writes Pins tab
8. Shows the 3 runes <-----	7. Returns JSON result	

Every single action (rolling, transforming, crafting, generating a code, granting a rune) follows this same shape: the browser sends a small JSON message describing what it wants to do, the Apps Script backend is the only thing that actually checks the rules and touches the spreadsheet, and it sends back the student's updated, correct data. **The browser never edits the spreadsheet directly and never decides on its own whether an action is legal** — it just displays whatever the backend says is true.

2.1 Why it's built this way (not, say, a normal database)

- **No server to pay for or maintain.** Google Apps Script and Google Sheets are free, and Google handles hosting, uptime, and security patching.
- **The teacher already owns and can read the data** at any time, just by opening the Sheet — no separate admin tool needed to inspect or manually fix something.
- **Good enough concurrency for a classroom.** A short server-side lock (Section 5.5) makes sure two students acting at the exact same instant can't corrupt each other's data.

3. The three kinds of credentials

It's easy to confuse these three, since they're all short numeric codes. They are completely separate and serve different purposes:

Credential	Length	Who has it	What it's for	Where it's checked
Milestone code	6 digits	The whole class shares codes handed out by the teacher	One-time reward: redeeming it grants 3 random runes, then it's burned	Pins tab (Used column flips to TRUE)
Adventurer PIN	4 digits	Each student sets their own, for their own name	Proves "this is really my account" before rolling, transforming, or crafting	Students tab (PinHash column, salted & hashed)
Teacher passphrase	Any length ≥ 3	The teacher only, shared across the whole deployment	Unlocks the Teacher tab: generating codes, granting God Runes, resetting a PIN	Config tab (TeacherPass row)

The adventurer PIN is the newest addition, built specifically so students can't tamper with each other's runes just by typing a classmate's name. The first time any name is used to roll, transform, or craft, the app asks that person to choose a 4-digit PIN — from then on, that name always asks for it again (once per browser session, since it's cached briefly so students aren't re-typing it constantly). *Viewing* a name's inventory does not require the PIN; only the three actions that change rune counts do.

4. Glossary, for anyone non-technical

Term	Plain-language meaning
Frontend	The part of the app you actually see and click on — the webpage itself.
Backend	The part running behind the scenes that enforces the rules and stores data. Students never see it directly.
API / endpoint	A specific "request type" the backend understands, like "roll runes" or "generate a code". The frontend sends one of these every time it needs something done.
Google Apps Script	A free tool from Google that lets you write small programs that live inside a Google Sheet or Doc and can be reached over the internet.
Deployment	A published, stable version of the backend program with its own web address. Editing the code does nothing to the live app until you deploy a new version.
Hash (of a PIN)	A one-way scrambled version of a PIN. The real 4-digit number is never stored or sent back anywhere — only this scrambled fingerprint, which can confirm a guess is correct without ever revealing the original number.
Lock (concurrency lock)	A brief "hold on, someone else is mid-edit" pause the backend uses so two students acting at the same instant can't corrupt each other's data.
localStorage / sessionStorage	Small amounts of data a website can remember inside one specific browser. Used here for convenience settings, never for the actual rune data.

5. The data model (the Google Sheet)

The Sheet has three tabs. Each row in Students or Pins is one record, exactly like a row in any spreadsheet.

5.1 Students tab — one row per adventurer

Column	Contains
Name	The adventurer's name, exactly as typed. Matching is case-insensitive.
TIR, ITH, NEF, TAL, ORT, THUL, SHAEL, IO, LUM, JAH, BER, ZOD	One column per ordinary rune, holding how many of that rune this student currently owns.
GOD	How many God Runes this student owns (hand-granted only, never rolled).
Crafted	A JSON list of every runeword this student has forged, each with a name and timestamp. e.g. [{"name":"Enigma","at":"2026-09-18T..."}]
PinHash	A salted SHA-256 fingerprint of the student's 4-digit PIN. Never the PIN itself.
PinSalt	A random value mixed into the PIN before hashing, unique per student, so two students with the same PIN don't produce the same hash.
PinFailCount	How many wrong PIN attempts in a row. Resets to 0 on a correct guess.
PinLockUntil	If set, a timestamp before which this student's PIN is temporarily locked out (after 5 wrong attempts in a row).

5.2 Pins tab — one row per milestone code ever generated

Column	Contains
Code	The 6-digit one-time code.
Used	TRUE once someone has redeemed it, otherwise FALSE.
UsedBy	Which adventurer name redeemed it.
CreatedAt / UsedAt	Timestamps for when the teacher generated it and when it was redeemed.

5.3 Config tab — small key/value settings

Currently holds a single row: TeacherPass → the shared teacher passphrase, set the first time anyone unlocks the Teacher tab.

5.4 Important: game data is duplicated on purpose

The list of runes (their names, rarity, and point values) and the list of runeword recipes exists **twice**: once in the frontend file (to draw the Codex tab, show craft cards, etc.) and once in the backend (as the actual source of truth used to validate every roll, transform, and craft). This is intentional — the frontend can never be trusted to enforce rules, since a student could edit it in their browser's developer tools. **But it means that any change to the runes or runewords must be made in both files, or the two will disagree.** See Section 9 for exactly where.

5.5 How concurrency is handled

Every backend action that changes data (rolling, transforming, crafting, generating a code, etc.) wraps its work in a LockService lock. If two students happen to click at the exact same moment, the second request simply waits its turn (up to 10 seconds) rather than the two writes colliding and corrupting a rune count.

6. How a roll actually gets validated, step by step

Using "Roll" as the example, since it touches every kind of check the backend does:

- The frontend sends the adventurer's name, their 4-digit PIN, and the 6-digit milestone code.
- The backend finds (or creates) that student's row in the Students tab.
- It checks the PIN: hashes the submitted PIN with that student's stored salt, and compares it to the stored hash. Wrong guesses increase a fail counter; 5 in a row locks that student out for 60 seconds.
- It checks the milestone code: does it exist in the Pins tab, and is its Used column still FALSE?
- Only if both checks pass does it roll three random runes (weighted by the rarity table in Section 7), add them to the student's inventory, and mark the milestone code as used.
- It returns the three runes drawn and the student's brand-new totals, which the frontend then displays.

Transform and Craft follow the same PIN-check-first pattern, just replacing the milestone-code check with their own rule (respectively: "do the selected runes add up to enough points?" and "does the student own every rune the recipe needs?").

7. Game data reference

7.1 The twelve runes

Rune	Roll odds	Point value (for Transform)
Tir	14%	2
Ith	14%	2
Nef	12%	5
Tal	12%	5
Ort	10%	9
Thul	10%	9
Shael	8%	14
Io	8%	14
Lum	4%	20
Jah	4%	20
Ber	2%	27
Zod	2%	27

The God Rune is a thirteenth, separate kind: it is never rolled (its odds aren't part of the 100% above) and can only be added to a student's inventory by the teacher, via "Grant a God Rune" in the Teacher tab. Two runewords (Summarmækir and Ran's Net) require one.

7.2 How a roll is weighted

A roll draws a random number between 0 and 100, then walks down the rune list subtracting each rune's percentage until the number goes negative — whichever rune it lands on is the result. This happens three independent times per Roll action, so duplicates are possible (e.g. two Tir runes in one roll).

7.3 Transform rule

To forge one target rune, a student destroys any combination of runes from their own inventory whose point values (Section 7.1) add up to **at least** the target rune's own point value. Left-over value is not returned — e.g. destroying 28 points of runes to forge a 27-point Ber rune is allowed, but the extra point is simply lost.

7.4 Runeword recipes

Every runeword requires owning one of each rune listed, all at once; crafting destroys exactly those runes.

Runeword	Slot	Recipe
Enigma	Chest Armor	Ber + Nef + Thul + Ort

Runeword	Slot	Recipe
Cling	Gloves	Io + Nef + Ith
Prism	Necklace	Tir + Nef + Tal
Silver Tongue of Rizz	Eyewear	Shael + Io
Quicksilver	Harness / Bridle	Lum + Ort + Ith
Charger	Harness / Bridle	Ber + Thul + Tal + Shael
Precious	Ring	Zod + Lum + Thul
Hive	Boots	Ber + Lum + Ort + Ith
Starboard	Oar / Paddle	Io + Nef + Tir
Blade of Carotene	Sword	Io + Nef + Tir
Folly	Scarf	Zod + Shael + Ort
Insolence	Weapon	Lum + Shael + Ort + Tal
Sage	Earrings	Tir + Nef + Ort + Shael
Lark	Scarf	Ith + Tal + Thul + Io
Inferno	Dish	Zod + Io + Thul
Socks of Blocks	Socks	Zod + Jah + Io + Tal
Invigorated by the North	Pendant / Necklace	Lum + Shael + Tal
Bramble	Leg Armor	Shael + Ort + Nef + Tir
Crescent Moon	Musical Instrument	Ber + Shael + Tal + Ith
Harmony	Head Armor	Lum + Io + Tal + Tir
Summarmækir	Weapon	1 God Rune + Zod + Jah + Io
Ran's Net	Weapon	1 God Rune + Ber + Lum + Shael
Herofeller	Scabbard	Tir + Tal + Ort + Io
Coolth	Pillow	Tir + Thul + Shael + Lum
Amulet of Misfortune	Necklace	Tir + Shael + Thul + Nef
Stuffed Buddy	Doll	Ith + Nef + Thul + Io
Sour Key	Food	Shael + Io + Jah + Ber, ×1 (Small) / ×2 (Medium) / ×3 (Large)

8. Full API reference

The frontend talks to the backend with a single web address (the Apps Script deployment URL). Every request is an HTTP POST with a JSON body containing an `action` field and whatever parameters that action needs. Every response is JSON shaped as either `{"ok":true,"data":...}` or `{"ok":false,"error":"..."}`.

action	Parameters	What it does	PIN required?
checkName	name	Reports whether this name already has a PIN set (used to decide whether to show the “set a PIN” or “enter your PIN” prompt).	No
claimPin	name, authPin	Sets a brand-new 4-digit PIN for a name that doesn't have one yet.	No
verifyPin	name, authPin	Checks a PIN is correct, without doing anything else.	—
getStudent	name	Returns a student's current inventory and crafted-items list. View only.	No
roll	name, code, authPin	Redeems a 6-digit milestone code for 3 random runes.	Yes
transform	name, target, selection, authPin	Destroys the chosen runes (selection) to forge one target rune.	Yes
craft	name, itemId, authPin	Destroys a runeword's recipe runes and adds the item to Crafted.	Yes
craftSour	name, mult, authPin	Same as craft, but for the size-scaling Sour Key (mult is 1, 2, or 3).	Yes
teacherSetup	pass	Sets the shared teacher passphrase (only works once, when none is set).	—
teacherLogin	pass	Checks the teacher passphrase is correct.	—
teacherGeneratePin	pass	Creates a new, unused 6-digit milestone code.	—
teacherGrantGod	pass, name	Adds one God Rune directly to a student's inventory.	—
teacherListPins	pass	Returns every milestone code ever generated and its redemption status.	—
teacherListStudents	pass	Returns every student's totals, crafted count, and whether a PIN is set.	—
teacherResetPin	pass, name	Clears a student's PIN so they can set a new one.	—
teacherReset	pass	Wipes all student and milestone-code data. Irreversible.	—

All “teacher...” actions require the correct pass value in place of a student PIN; there is no separate teacher session token — the passphrase is simply re-sent with every teacher request.

9. Notes for future Claude sessions

If you (a future instance of Claude) are being asked to modify this app, here is what you need to know before touching anything.

9.1 Files and where they live

File	Role	How to change it live
runic-system-cloud.html	The real, multi-device frontend. Single self-contained file: HTML, CSS, and JavaScript all inline, no build step.	Edit the file directly, then re-upload/overwrite it wherever the teacher hosts it (e.g. illswap.net). No redeploy needed — it's a static file.
Code.gs	The backend. Lives inside a Google Apps Script project bound to the teacher's Google Sheet.	Edit the code in the Apps Script editor, then Deploy → Manage deployments → edit → New version → Deploy. Editing alone does NOT affect the live app.
runic-system.html	A legacy, fully offline fallback version that stores everything in browser localStorage instead of the Sheet. No multi-device sync. Kept only as a no-setup emergency option.	Standalone; unrelated to the two files above. Only touch if asked.

9.2 Rules to respect when changing game content

- If you add, remove, or change a rune or a runeword recipe, you must edit it in **both** runic-system-cloud.html (the RUNES / RUNEWORDS arrays near the top of the `<script>` block) **and** Code.gs (the same two constants near the top of the file). They must stay identical in rune codes, point values, and recipes, or the frontend's craft-card display will disagree with what the backend actually allows.
- After any Code.gs edit, remind whoever is deploying that a **new deployment version** is required (Section 9.1) — this is the single most common way a fix silently "doesn't work" for the teacher.
- Never move game-rule validation (roll odds, transform math, craft requirements) into the frontend only. The backend must remain the source of truth, since the frontend is fully visible and editable by anyone with browser developer tools.
- The Sheet's column headers are load-bearing: both `readStudentPublic_/readStudentAuth_` in Code.gs and the `writeInventoryDelta_/writeFields_` helpers look columns up *by header name*, not position. Renaming a header breaks the mapping. Adding a new header is safe (`ensureColumns_` appends without disturbing existing ones).

9.3 What NOT to do

- Do not publish runic-system-cloud.html through Claude's Artifact tool as the "live" version — Claude's hosted artifact preview blocks outbound network calls to external domains like `script.google.com`, so the backend connection will silently fail there. This file must be opened as a real file (self-hosted, or opened locally) to function.

- Do not store the real 4-digit PIN or the teacher passphrase anywhere except as intended (PinHash/PinSalt for PINs, plain in Config for the teacher passphrase, which is an accepted, documented trade-off for a classroom-grade tool, not a security oversight).
- Do not remove the LockService locks around mutating Code.gs functions; they are the only thing preventing two simultaneous requests from corrupting a rune count.

9.4 Good first questions to ask the teacher before changing something

- Is this a game-content change (new rune, new runeword, new odds) or a behavior change (new feature, new screen, new validation rule)? Content changes need edits in two files; behavior changes are usually one file.
- Does this change need to preserve existing student data? (Almost always yes — avoid anything that would require wiping the Students or Pins tabs.)
- Will the teacher need to redeploy the Apps Script, and do they still remember how (Section 9.1)?

9.5 Known limitations, by design

Limitation	Why it's acceptable here
Google Sheets isn't built for heavy concurrent traffic.	Fine for a class of dozens acting occasionally; would need a real database if this ever scaled to hundreds of simultaneous users.
The teacher passphrase and student names travel as plain JSON over HTTPS.	Encrypted in transit by HTTPS; acceptable for a fun in-class tool with no sensitive personal data, not designed for anything higher-stakes.
Any two people can independently create an account under the same name	if the second person gets there before the first sets a PIN. First to set a PIN "owns" that name from then on — an accepted trade-off for keeping the system simple and account-free.
No true user accounts (email/password, etc.).	Deliberately avoided so any student can start playing immediately with just a name, no sign-up flow.

10. One-paragraph mental model

If you remember only one thing from this document:

The webpage is just a display. Every rule — who can act, what a roll produces, what a craft costs — lives in Code.gs, which is the only thing allowed to read or write the Google Sheet. The Sheet is the entire database. Nothing is real until Code.gs says it's real.

Document generated for the teacher's ongoing use with Claude. Keep this alongside runic-system-cloud.html and Code.gs, and share it at the start of any future conversation asking Claude to modify the Runic System.